Circles Invitation At Scale Audit

Gnosis Ltd - Report by Côme du Crest

2025-07-11

Table of contents

- Table of contents
- Circles Invitation At Scale Audit
  - Scope
  - Context
  - Status

Circles Invitation At Scale Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within `aboutcircles/circles-invitation-at-scale` as of commit `0x0660efc`.

Context

ReferralsModule

`ReferralsModule` can create accounts that will create a safe proxy and set the storage mapping `accounts[signer].account` to the created safe proxy address. Only the `InvitationModule.GenericCallProxy` can create accounts. The safe can later be claimed by proving ownership of the signer private key which will set-up the WebAuthn passkey for the safe user and disable the `ReferralsModule` on the safe.

The safe is configured as follows:

- Sole owner is `SafeWebAuthnSharedSigner`
- Threshold is 1
- Modules are `Safe4337Module`, `InvitationModule`, and `ReferralsModule`
- FallbackHandler is `Safe4337Module`

The only way to execute transactions on the safe is through modules or if they are signed (validated) by the owner `SafeWebAuthnSharedSigner`.

To validate transactions `SafeWebAuthnSharedSigner` must be configured by setting the data for WebAuthn signer. This can only be performed via a delegatecall from the safe. `ReferralsModule` configures the `SafeWebAuthnSharedSigner` on a safe when it is claimed and accounts cannot be claimed twice. The owner and `ReferralsModule` cannot be used to miss-configure / poison the safe.

The module `Safe4337Module` executes transactions on the safe only on `msg.sender` and only in functions guarded with `onlySupportedEntryPoint`. The `onlySupportedEntryPoint` verifies that the last 20 bytes of calldata correspond to the supported ERC4337 entrypoint. These bytes are populated by the safe fallback manager to `msg.sender` before calling the fallback handler.

As such `msg.sender` is enforced to be the entrypoint contract. As such, only the entrypoint can make `Safe4337Module` execute on a transaction on the safe. This should only be possible if the entrypoint called `Safe4337Module.validateUserOp()` which verifies the signatures on the userOp by the safe, which is only possible if `SafeWebAuthnSharedSigner` has been configured. As a result `Safe4337Module` cannot be used to miss-configure / poison the safe.

The module `InvitationModule` only executes transactions on the safe with the circles' Hub as target. It cannot be used to miss-configure / poison the safe.

For these reasons I believe account creation / claim via `ReferralsModule` is safe.

GenericCallProxy

`GenericCallProxy` executes generic call to any target when called by its owning module (deployer). It returns an address or list of addresses as decoded from the return value of the executed call.

The `InvitationModule` creates a new `GenericCallProxy` upon deployment. That proxy is the sole contract able to call `ReferralsModule.createAccount()` and `ReferralsModule.createAccounts()`.

The `InvitationFarm` uses the same `GenericCallProxy` as `InvitationModule`. That module will be allowed to create new bots through the farm via the proxy.

InvitationModule

This module can trust any address on the Hub through `trustInviter()`. Its most important function is `onERC1155Received()` which does the following:

- Sets non reentrant flag to `from` (token transfer origin)
- Ensures values is invitation fee
- Validate `from` is a human
- Sets the `inviter` to token id
- Calls the generic proxy if `data.length > 32` to get the `invitee`, otherwise parse data as address
- Does some unintelligible verifications/enforcements regarding humanity, trusts, and enabled modules
- Makes `inviter` invite `invitee` by transferring invitation fee to `inviter` and making `invitee` register on the hub through `_callHubFromSafe`

To my understanding, the data provided to the function may be a call to `ReferralsModule.createAccount()` which will deploy a safe proxy with the invitation module enabled and return its

address. Alternatively, the data provided may be a call to `InvitationFarm.createBots()` to create new bots when originating from `InvitationFarm.growFarm()`.

The function `onERC1155BatchReceived()` has a very similar function for a list of invitees albeit with slightly different shenanigans in case `inviter` is `from` and does not have the invitation module enabled.

The functions `onERC1155Received()` and `onERC1155BatchReceived()` have the side-effect that `inviter` trusts any `invitee` address provided by the `from` user for a single block. This is not a problem since that `invitee` must be registered as a human on the same block and must not have been registered beforehand so it will only hold the invitation bonus which is less than what `from` spent in `inviter` token. As a result, the `invitee` could rip off `inviter` of less than what `from` lost.

InvitationBot

`InvitationBot` is a proxy that allows the invitation farm or its invitation module to execute any call. It only accepts to receive its own token from the hub in `onERC1155Received()`.

InvitationFarm

This contract can create a large amount of `InvitationBot` to farm CRC. The farm is grown through `growFarm()` which loops over bots to transfer the invitation fee from each bot to the invitation module. The encoded data passed to `InvitationModule.onERC1155BatchReceived()` is a call to `InvitationFarm.createBots()` which is authorized because the invitation module uses the same generic call proxy as the invitation farm and the origin inviter is set in the invitation module. Once the new bot is created, the invitation module will ensure the bot is registered as a human on the hub.

The CRC from the bots can be claimed through `claimInvites()` which transfer CRCs to an authorized `msg.sender`.

Status

Some comments have been discussed directly with the developer. No issues have been found.